

System Analysis & Design

Long-Answer Question Bank: Week 1 – Week 8

Broad Questions (7–8 Marks Each) with Full Model Answers — Organized Week by Week

Course: CSE0613-2201 System Analysis & Design
University of Global Village (UGV)

WEEK 1 & 2 — Systems Analysis & Design Basics

Part A — Technical / Theory Questions

Each question carries 7–8 marks.

Q1. Explain the systems development process and comprehensively differentiate between System Analysis and System Design. Support your answer with a suitable example.

[7–8 Marks]

Answer: Systems development is a systematic process consisting of five broad phases: Planning, Analysis, Design, Deployment (Implementation), and Maintenance. Within this process, System Analysis and System Design are the two most critical and frequently confused phases.

System Analysis is the process of collecting and interpreting facts, identifying problems, and decomposing an existing or proposed system into its components in order to understand its objectives. It is essentially a problem-solving technique that ensures every component of a system works efficiently toward its purpose. Analysis is concerned with *what* the system should do — it produces a requirements document as its key output, using techniques such as interviews, surveys, observation, and document analysis, and results only in a high-level understanding of the problem domain.

System Design, in contrast, is the process of planning a new system or replacing an existing one by defining the components or modules needed to satisfy the specific requirements gathered during analysis. Design focuses on *how* the system's objectives will be accomplished, producing detailed system design specifications using techniques such as UML, Data Flow Diagrams, and Entity-Relationship Diagrams.

Example: Consider a university replacing its manual admission process. During analysis, the team studies the current paper-based workflow, interviews staff and students, and documents requirements such as "applicants must be able to apply and check status online." During design, the team then produces the database schema, screen layouts, and server architecture needed to implement those requirements. Without analysis, the design would risk solving the wrong problem; without design, the requirements would never be translated into a working system.

Q2. Describe, with suitable real-life examples, the five properties that every system possesses, and explain why these properties matter for the successful functioning of a system.

[7–8 Marks]

Answer: Every system — whether an organization, a machine, or a piece of software — exhibits five defining properties.

1. Organization refers to the structure and order in the arrangement of components that helps achieve predetermined objectives. For example, a company is organized into departments such as sales, finance, and HR, each with a defined role.

2. Interaction is the manner in which components operate with each other. For instance, in a manufacturing company, the purchasing department must interact with the production department to ensure raw materials arrive on schedule.

3. Interdependence means components rely on one another's output as their own input. In an e-commerce business, the warehouse cannot dispatch an order until the payment subsystem confirms that payment has

been received.

4. Integration is concerned with how the parts of a system are connected so that they work together as a whole even though each performs a unique function — for example, a hospital's admissions, billing, and pharmacy systems must be integrated so a patient's record flows seamlessly between them.

5. Central Objective means the system must have one clear, shared purpose known to all users; for example, all departments of an airline ultimately serve the central objective of safely and efficiently transporting passengers.

These properties matter because a system lacking any one of them — e.g., poor integration or an unclear central objective — tends to produce conflicting priorities, duplicated effort, and ultimately, systems that fail to meet stakeholder needs. Recognizing these properties during analysis and design helps ensure a cohesive, well-functioning final system.

Q3. "Analysis specifies what a system should do, while design specifies how it should do it." Critically discuss this statement, explaining the consequences of not following this sequence in a real software project.

[7–8 Marks]

Answer: This statement captures the fundamental division of labor between the two core phases of systems development. System Analysis is problem-oriented: it studies the current situation, gathers stakeholder requirements, and defines objectives without committing to any particular technical solution — answering the question of *what* outcomes the system must deliver. System Design is solution-oriented: once requirements are known, design translates them into concrete architecture, data structures, interfaces, and modules — answering *how* those outcomes will be technically achieved.

This sequence exists because design decisions should be traceable back to validated requirements; skipping or shortcutting analysis means designers are essentially guessing at what the users need. In a real software project, if a team begins designing database schemas and interfaces before requirements are properly gathered and confirmed, several consequences typically follow: (1) the finished system may technically function well but fail to solve the actual business problem; (2) expensive rework becomes necessary once the missing requirements surface late in the project; (3) stakeholders lose confidence as the delivered system does not match their expectations; and (4) project timelines and budgets are extended well beyond original estimates. Therefore, while analysis and design are closely interconnected — with the requirements document from analysis serving as the direct input to design — the two must remain distinct, sequential phases for a project to reliably succeed.

Part B — Scenario-Based Questions

Each question carries 7–8 marks.

Q1. A small college wants to computerize its examination result processing system, which is currently handled manually using registers and calculators. As the appointed system analyst, describe in detail how you would carry out the analysis phase and the design phase for this project, and explain what properties of a system you would specifically look out for.

[7–8 Marks]

Answer: As the system analyst, I would begin with the **analysis phase** by using fact-finding techniques: interviewing exam-cell staff and teachers to understand the current manual process, observing how marks are currently compiled and results generated, and examining existing registers and mark sheets (document

analysis). From this, I would identify problems such as calculation errors, delays in result publication, and difficulty in generating transcripts. I would then document requirements such as "the system must automatically calculate GPA," "staff must be able to enter marks online," and "students must be able to view results securely." This becomes the requirements document, the key output of analysis.

In the **design phase**, I would translate these requirements into a concrete solution: designing a database schema (ERD) linking Students, Courses, and Marks; designing the architecture (e.g., a web application with a secure results portal); designing the interface for teachers to enter marks and for students to view results; and designing the algorithm for GPA calculation.

Throughout both phases, I would specifically watch for the system's **properties**: Organization (a clear structure separating data entry, calculation, and result publication), Interaction (how the exam cell's data entry interacts with the academic office's approval step), Interdependence (results cannot be published until all subject marks are entered), Integration (the results module must connect smoothly with the existing student database), and a clear Central Objective (accurate, timely, and secure result processing) that all stakeholders agree on before development begins.

Q2. A manufacturing company's various departments — procurement, production, quality control, and sales — operate almost entirely in isolation, rarely sharing data, which has led to frequent stock shortages and conflicting priorities. Using the concept of system properties, analyze what is going wrong and recommend how a new integrated information system should be designed to fix it.

[7–8 Marks]

Answer: The scenario reveals a clear absence of key system properties. First, there is a lack of **Integration**: although each department performs a distinct function, they are not connected as a unified whole, so information generated in one area (e.g., a shortage flagged by production) never reaches another area (e.g., procurement) in time. Second, there is insufficient **Interaction**: departments are not properly communicating despite their work affecting one another. Third, the company appears to lack a clearly understood **Central Objective** shared across departments, since each is optimizing for its own local goals (e.g., sales pushing for more orders without checking production capacity) rather than the organization's overall success. Fourth, **Interdependence** is not being respected: production's output requirements depend on procurement's input of raw materials, but this dependency is not currently reflected in how the departments operate.

To fix this, a new integrated information system should be designed with a shared database that all departments can access in real time, so that a stock shortage flagged in production automatically triggers a procurement alert (restoring Interdependence and Interaction). The system's modules (procurement, production, quality control, sales) should be architecturally integrated rather than siloed, with a single dashboard reflecting the company's shared objective — e.g., on-time order fulfillment. During analysis, the analyst should explicitly document each department's dependencies on the others so the design phase can build the necessary data flows and notifications into the system architecture.

WEEK 3 & 4 — Elements, Types of Systems & Categories of Information

Part A — Technical / Theory Questions

Each question carries 7–8 marks.

Q1. Explain, in detail, the six elements of a system. Illustrate each element using a single real-world system of your choice.

[7–8 Marks]

Answer: Every system, regardless of its domain, is composed of six core elements, which can be clearly illustrated using an **online banking system** as a running example.

1. Inputs and Outputs: Inputs are the information entering the system for processing — in online banking, this includes login credentials and a fund-transfer request. Output is the useful outcome produced — the completed transaction and updated balance.

2. Processor(s): The processor transforms input into output. In online banking, this is the core banking software that validates the transfer request, checks balance sufficiency, and executes the transaction.

3. Control: This is the decision-making subsystem guiding input, processing, and output, informed by feedback. If a user attempts to transfer more money than their daily limit allows, the system rejects the transaction (negative feedback), while a successfully completed low-risk transaction may simply proceed (positive/routine feedback).

4. Feedback: Closely tied to control, feedback can be positive (routine, reinforcing correct operation, such as confirming a successful login) or negative (informational, prompting corrective action, such as flagging a suspicious login attempt).

5. The Environment: This is the "super-system" within which the bank's system operates — including banking regulations, the wider financial network, and competing banks — all of which impose constraints the system must respect (e.g., mandatory two-factor authentication under regulation).

6. Boundaries and Interface: The boundary defines what is inside the online banking system (e.g., the mobile app and its servers) versus outside (e.g., a third-party payment gateway). The interface is the point of connection — such as the API through which the banking app communicates with the national payment settlement network.

Q2. Discuss the different ways in which systems can be classified. Provide one clear example for each classification type discussed.

[7–8 Marks]

Answer: Systems can be classified along six major dimensions.

Physical or Abstract: Physical systems are tangible, such as a company's server room, while abstract systems are conceptual, such as a financial forecasting formula.

Open or Closed: Open systems interact with their environment, receiving inputs and sending outputs — for example, a retail business responding to seasonal demand. Closed systems are isolated from environmental influence, such as a sealed laboratory experiment; truly closed systems are rare in reality.

Adaptive or Non-Adaptive: Adaptive systems respond to environmental change to improve performance, such as a human adjusting behavior to new circumstances, while non-adaptive systems, such as a basic mechanical device, do not respond to their environment.

Permanent or Temporary: Permanent systems persist over a long time, such as a country's legal system,

while temporary systems are built for a specific purpose and later dismantled, such as the staging setup for a music concert.

Natural or Manufactured: Natural systems are created by nature, such as the solar system, while manufactured systems are man-made, such as a railway network.

Man-Made Information Systems: These are interconnected sets of information resources — hardware, software, data, and communication — managed under an organization's direct control, such as a company's payroll or enterprise resource planning (ERP) system.

Understanding these classifications helps analysts and designers choose appropriate strategies — for instance, a temporary or closed system requires far less investment in long-term adaptability than an open, permanent one.

Q3. Explain the three categories of information used in organizations, the management level each serves, and the type of system typically used to produce each. Discuss why this categorization is important for effective decision-making.

[7–8 Marks]

Answer: Organizational information is typically categorized into three levels, aligned with the structure of management.

Strategic Information is required by top-level management for long-range planning, typically spanning several years — examples include revenue trends, financial investment outlooks, and population growth projections. This information is generated with the aid of a **Decision Support System (DSS)**, which helps executives evaluate complex, less-structured decisions.

Managerial Information is required by middle management for short- to intermediate-range planning, typically measured in months — examples include sales analysis, cash-flow projections, and annual financial statements. This is achieved through a **Management Information System (MIS)**, which aggregates operational data into summarized reports.

Operational Information is required by lower-level management for daily and short-term planning to run day-to-day activities — examples include employee attendance records, overdue purchase orders, and current stock levels. This is produced by a **Data Processing System (DPS)**, which handles routine, high-volume transactional data.

This categorization is important because it ensures that decision-makers at every level receive information at the right level of detail and time horizon: overwhelming a CEO with raw daily transaction logs would obscure the big picture, while giving a store supervisor only five-year strategic trends would be useless for today's operational decisions. Proper categorization therefore improves both the efficiency and the accuracy of organizational decision-making at every level.

Part B — Scenario-Based Questions

Each question carries 7–8 marks.

Q1. An online food-delivery company is reviewing its platform's architecture. Using the concept of system elements, identify and explain each of the six elements as they apply to this food-delivery platform, and explain how the elements work together during a single order.

[7–8 Marks]

Answer: Applying the six elements of a system to the food-delivery platform:

Input: The customer's order details (items, address, payment method) entered through the app.

Processor: The platform's backend logic, which matches the order to a nearby restaurant, calculates delivery time, and assigns a delivery rider.

Output: The successfully delivered meal and the order confirmation/receipt sent to the customer.

Control (with Feedback): The system's rules that, for example, reject an order if the restaurant is outside the delivery radius (negative feedback, prompting correction) or automatically confirm and proceed when all conditions are met (positive/routine feedback).

Environment: External factors such as traffic conditions, food-safety regulations, and competing delivery apps, all of which influence how the platform must operate.

Boundaries and Interface: The boundary separates the platform's own systems (app, order database) from external systems such as the restaurant's POS system or the payment gateway; the interface is the API connecting them.

During a single order, these elements work together sequentially: the input (order) is passed to the processor, which uses control logic (checking radius, restaurant availability) informed by feedback to either proceed or reject; environmental factors like traffic inform delivery-time estimates; and the final output (delivered meal) is produced through the interface connecting the app, restaurant, and payment systems.

Q2. A hospital is planning to introduce (a) a portable diagnostic kiosk used only during an annual one-day health camp, and (b) a permanent hospital-wide patient information system. Classify each system using the classification types discussed in class, and explain what different levels of information (strategic, managerial, operational) each system might feed into.

[7–8 Marks]

Answer: (a) The portable diagnostic kiosk used only for a one-day annual health camp is a **Temporary** system (built for a fixed, short-duration purpose and dismantled afterward), a **Physical** system (a tangible device), and a **Manufactured** system (man-made). It is also arguably an **Open** system, since it takes in patient data from the environment during the camp. The information it generates is mostly **Operational** — e.g., today's patient count and immediate test results used by camp staff — though aggregated data afterward (e.g., disease prevalence trends in the community) could feed into **Managerial** or even **Strategic** planning if the hospital uses it to decide whether to expand outreach programs.

(b) The permanent hospital-wide patient information system is a **Permanent** system (intended to operate indefinitely), an **Open** system (continuously interacting with patients, staff, and external labs), and a **Man-Made Information System** (an interconnected set of hardware, software, and data resources under hospital management). It generates information across all three levels: **Operational** (daily patient admissions, bed occupancy), **Managerial** (monthly department performance, resource utilization reports for department heads), and **Strategic** (multi-year trends in patient volume used by hospital administration to decide on expansion or new specialty departments).

WEEK 5 & 6 — Analysis vs. Design, Role & Qualities of the Analyst

Part A — Technical / Theory Questions

Each question carries 7–8 marks.

Q1. Provide a comprehensive comparison between System Analysis and System Design in terms of focus, output, techniques used, and level of detail, and explain how the two phases relate to one another.

[7–8 Marks]

Answer: System Analysis and System Design, while part of the same development lifecycle, differ significantly across four dimensions.

Focus: Analysis focuses on understanding the problem domain and gathering requirements — identifying stakeholders, their needs, and the scope of the system. Design focuses on specifying the solution — translating those requirements into a technical blueprint.

Output: The primary output of analysis is a requirements document, capturing what the system must achieve. The primary output of design is a set of system design specifications, detailing architecture, data structures, and interfaces.

Techniques: Analysis relies on fact-finding techniques such as interviews, surveys, observation, and document analysis. Design relies on modelling techniques such as UML, Data Flow Diagrams (DFDs), Entity-Relationship Diagrams (ERDs), decision trees, and state transition diagrams.

Level of Detail: Analysis produces a high-level understanding of the problem, while design produces detailed, implementable specifications.

Despite these differences, the two phases are closely interconnected: the output of analysis (the requirements document) directly serves as the input to design, and design specifications must always be validated against the original requirements to ensure the final system genuinely meets user and stakeholder needs. In this sense, analysis and design form a continuous, traceable chain rather than two isolated activities.

Q2. Discuss the role of the system analyst within an information systems project, and describe the key qualities that make an analyst effective in this role.

[7–8 Marks]

Answer: The system analyst occupies a central, multi-faceted role in any information systems development project. Fundamentally, the analyst must understand how to apply technology to solve business problems — bridging the gap between what an organization needs and what technology can deliver. Beyond this technical bridging role, the analyst frequently serves as a **change agent**, proactively identifying opportunities for organizational improvement that go beyond the immediate scope of a single system, and helping guide the organization through the resulting changes in process and culture.

To perform this role effectively, an analyst needs several key qualities. First, they must be a strong **problem solver**, capable of diagnosing root causes rather than just symptoms. Second, they must be an excellent **communicator**, since they constantly translate between non-technical stakeholders (who describe needs in business language) and technical teams (who require precise specifications) — miscommunication at this stage is one of the leading causes of project failure. Third, they must maintain strong **personal and**

professional ethics, since they routinely handle sensitive organizational and personal data and must be trusted to act in stakeholders' best interests, even when it conflicts with a client's stated preference. Fourth, they must be **self-disciplined and self-motivated**, as systems projects are often long, ambiguous, and require sustained independent effort without constant supervision. Together, these qualities allow the analyst to reliably deliver systems that genuinely solve the organization's problems.

Q3. Explain the key activities involved in the system analysis phase and describe how each activity contributes to the successful completion of a project.

[7–8 Marks]

Answer: System analysis comprises five key activities that build on one another sequentially.

1. Requirement Gathering involves identifying the needs and expectations of users and stakeholders, typically through interviews, surveys, and observation. This activity forms the raw material for everything that follows.

2. Requirement Analysis takes the gathered requirements and examines them for consistency, feasibility, and completeness — resolving contradictions between what different stakeholders have requested.

3. Feasibility Study assesses whether the proposed system is achievable from technical, economic, and operational standpoints, helping the organization avoid investing in a project that cannot realistically be delivered or sustained.

4. Process Modelling creates diagrams and models (such as early-stage DFDs) to represent both the current and proposed business processes, giving stakeholders a visual way to validate that the analyst has correctly understood their workflow.

5. Data Modelling defines the data entities, attributes, and relationships within the system, laying the groundwork for the database design that will occur in the design phase.

Together, these activities ensure that by the end of the analysis phase, the project team has a validated, feasible, and well-documented understanding of both what the system must do and what data it must manage — directly enabling a smoother, better-informed design phase to follow.

Part B — Scenario-Based Questions

Each question carries 7–8 marks.

Q1. During a project to build a new online tuition-payment portal for a school, a client insists on a specific payment workflow that the analyst believes will cause frequent transaction failures and parent complaints. Explain, step by step, how the analyst should professionally handle this situation, referencing the analyst's role and required qualities.

[7–8 Marks]

Answer: The analyst should approach this situation methodically, drawing on both their role as a problem-solver/change agent and their professional qualities.

Step 1 — Validate the concern: Before raising an objection, the analyst should confirm their concern is well-founded, perhaps by testing the proposed workflow logically or against similar real-world systems, drawing on their problem-solving skill.

Step 2 — Communicate clearly and diplomatically: The analyst should present the concern to the client using clear, non-technical language, explaining specifically why the proposed workflow risks transaction failures (e.g., a missing confirmation step before processing payment), demonstrating strong communication skills.

Step 3 — Propose an alternative: Rather than simply objecting, the analyst should recommend a revised workflow that meets the client's underlying goal (e.g., a simple payment process) while avoiding the identified risk — acting here as a change agent who improves the outcome beyond what the client initially envisioned.

Step 4 — Uphold ethical responsibility: If the client still insists on the original approach, the analyst has an ethical obligation to document the risk clearly in writing, ensuring the client makes an informed decision rather than silently complying with a flawed design purely to avoid conflict.

This step-by-step approach demonstrates how the analyst's qualities — problem-solving, communication, and ethics — work together to protect both the client's interests and the quality of the final system.

Q2. A software company builds a customer relationship management (CRM) system for a client but does not involve any analyst in the process. Developers work directly from a two-line email describing the client's needs. Six months later, the client rejects the system, saying it does not match their business processes at all. Diagnose what went wrong using the analysis vs. design framework and the role of the system analyst, and recommend how the project should be restarted correctly.

[7–8 Marks]

Answer: This project failure can be traced directly to the complete absence of a proper analysis phase and the corresponding lack of a system analyst's involvement. A two-line email cannot serve as a valid requirements document — it lacks the depth achieved through structured fact-finding techniques such as interviews, surveys, observation, and document analysis. Without an analyst gathering, analyzing, and validating requirements, the development team was forced to design and build the CRM system based on assumptions rather than an accurate understanding of the client's actual business processes. This directly explains why, after six months, the delivered system does not match reality: the design phase had no valid requirements document to work from, so its detailed specifications (however technically sound) targeted the wrong problem.

To correctly restart this project, the company should first bring in a system analyst to conduct a proper analysis phase from scratch: interviewing the client's staff across relevant departments, observing existing (even informal) customer-management workflows, and reviewing any existing documents or spreadsheets used to track customers. This should produce a formal requirements document, validated with the client, that clearly specifies workflows, data requirements, and success criteria. Only once this is approved should the design phase restart, producing new ER diagrams, interface designs, and architecture that are traceable directly back to the validated requirements — ensuring the rebuilt system actually reflects the client's real business processes this time.

WEEK 7 & 8 — Techniques, Design Activities & Scaling

Part A — Technical / Theory Questions

Each question carries 7–8 marks.

Q1. Compare and contrast the techniques used in System Analysis with the techniques used in System Design. For each technique, briefly explain its purpose and when it is most appropriately used.

[7–8 Marks]

Answer: System Analysis and System Design each rely on a distinct toolkit of techniques suited to their different goals.

Analysis techniques are fact-finding in nature: **Interviews** gather in-depth information directly from stakeholders and are best used when nuanced, flexible dialogue is needed; **Surveys** use questionnaires to reach many respondents efficiently, ideal for large or geographically spread populations; **Observation** involves watching the current system in operation, best used when stakeholders cannot easily articulate their own workflow; **Document Analysis** examines existing reports, forms, and manuals, useful for understanding an established process quickly; and **Prototyping** builds simplified mock-ups to gather early feedback, ideal when requirements are still unclear or evolving.

Design techniques are modelling-oriented: **UML** visualizes and documents software structure and behavior, useful for communicating architecture to developers; **Data Flow Diagrams (DFDs)** illustrate how data moves through a system's processes, useful for understanding data transformation; **Entity-Relationship Diagrams (ERDs)** represent data entities and their relationships, essential for database design; **Decision Trees** map out possible outcomes and decisions in a process, useful for modelling complex business rules; and **State Transition Diagrams** represent the states an object/system can be in and the transitions between them, ideal for modelling workflows with distinct lifecycle stages (e.g., an order's status).

In summary, analysis techniques are about discovering and confirming requirements from people and existing artifacts, while design techniques are about visually specifying a technical solution once those requirements are known.

Q2. Explain horizontal scaling and vertical scaling in detail, including their respective benefits and limitations, and discuss the key factors that should guide the choice between them for a growing system.

[7–8 Marks]

Answer: Scaling refers to adjusting a system's resources — computing power, storage, or network capacity — to handle increasing demand without sacrificing performance.

Vertical scaling (scaling up) means adding more resources, such as CPU or RAM, to a single existing server. Its benefits include simplicity (straightforward to implement), reduced operational complexity (only one machine to manage and monitor), stronger data consistency (since everything runs on one machine), and suitability for monolithic applications. Its limitations include being a single point of failure (if that server fails, the whole system goes down), hard physical resource limits (a server can only be upgraded so far), high cost at the high end, and potential downtime during hardware upgrades.

Horizontal scaling (scaling out) means adding more machines to share the workload. Its benefits include near-unlimited scalability (machines can keep being added as demand grows), fault tolerance (if one node fails, others continue serving traffic), cost efficiency at scale (using many lower-end machines rather than one expensive one), and natural fit for cloud-native and microservices architectures. Its limitations include increased management complexity, potential data-consistency challenges across distributed nodes, added network overhead from inter-node communication, and the possibility that the entire application stack must be redesigned to support distribution.

The choice between the two should be guided by: **application architecture** (monolithic systems scale more

naturally vertically, while distributed/microservices systems scale more naturally horizontally), **budget** (horizontal scaling is often more cost-effective in the cloud), **consistency requirements** (systems like banking that demand strict consistency may favor vertical scaling), and **expected growth** (rapidly growing applications benefit from horizontal scaling's theoretically unlimited capacity).

Q3. Describe the key activities involved in system design, and explain how these activities apply when designing a real-world system such as a hospital appointment-booking system.

[7–8 Marks]

Answer: System design comprises five key activities that collectively transform requirements into a detailed, implementable blueprint.

Architectural Design determines the overall structure and major components of the system. For a hospital appointment-booking system, this might mean deciding on separate modules for patient registration, doctor scheduling, and notifications.

Component Design designs individual components and how they interact — for example, defining exactly how the scheduling module checks a doctor's availability before confirming a booking.

Interface Design specifies the interfaces between components and with external systems — for example, how the booking system's mobile app communicates with the hospital's internal database, or how it sends SMS reminders through a third-party messaging API.

Data Design designs the database schema and data structures — for example, an Entity-Relationship Diagram linking Patients, Doctors, Appointments, and Departments.

Detailed Design creates detailed specifications for each component, including algorithms and data flow — for example, the exact logic used to prevent two patients from booking the same doctor's slot simultaneously.

Applied together, these activities ensure the hospital appointment-booking system is not only conceptually sound but fully specified down to the level of database tables, module interactions, and precise algorithms — ready for developers to implement without ambiguity.

Part B — Scenario-Based Questions

Each question carries 7–8 marks.

Q1. An e-commerce startup currently runs its entire application on a single powerful server. During flash sales, the server frequently crashes due to overwhelming traffic. As a system designer, propose and justify a complete scaling strategy for this company, including any trade-offs they should expect.

[7–8 Marks]

Answer: The startup's current single-server setup represents pure vertical scaling, which is precisely why it struggles during flash sales: a single point of failure combined with a hard resource ceiling means that once traffic exceeds the server's maximum capacity, the entire system goes down.

As the system designer, I would recommend transitioning toward **horizontal scaling**, since e-commerce platforms with unpredictable, spike-prone traffic are a textbook case favoring scale-out architectures. Concretely, this would involve: (1) breaking the monolithic application into smaller, independently scalable services (e.g., separate services for product catalog, cart, checkout, and payment) so that only the components under heavy load during a sale (like checkout) need to scale up; (2) deploying multiple instances

of each service behind a load balancer, so incoming traffic is distributed and no single machine is overwhelmed; and (3) using auto-scaling in the cloud so additional server instances are automatically added when traffic spikes and removed afterward to control cost.

The trade-offs the company should expect include: increased architectural and operational complexity (managing many services and servers instead of one), the need to carefully handle data consistency across distributed components (e.g., ensuring inventory counts remain accurate when multiple servers process orders simultaneously), and additional network overhead from inter-service communication. However, these trade-offs are generally justified by the reliability, fault tolerance, and near-unlimited scalability the company gains — directly solving the flash-sale crash problem.

Q2. You are asked to design a university library management system that must support book search, reservation, borrowing, returning, and overdue-fine tracking. Explain which system design techniques you would use for which parts of this system, and justify each choice.

[7–8 Marks]

Answer: For this library management system, different design techniques suit different aspects of the system.

For overall **data design**, I would use an **Entity-Relationship Diagram (ERD)** to model entities such as Book, Member, Loan, and Fine, and the relationships between them (e.g., a Member can have many Loans; each Loan references one Book). This is justified because ERDs are specifically designed to represent data entities and relationships, which is essential for designing the underlying database.

For modelling how a book moves through its lifecycle — Available → Reserved → Borrowed → Overdue → Returned — I would use a **State Transition Diagram**, since this technique is specifically suited to representing the distinct states an object (here, a book copy) can be in and the events that trigger transitions between them (e.g., "member borrows book" triggers the transition from Available to Borrowed).

For representing how data flows between the search interface, the database, and the notification system (e.g., automatically flagging overdue books), I would use a **Data Flow Diagram (DFD)**, since DFDs are designed to show the flow of data between functions of a system, giving a clear picture of how a search query moves from the user interface to the database and back.

Finally, for the overall structure of the system's modules (search module, borrowing module, fine-calculation module) and how they interact, I would use **UML** diagrams (such as class or component diagrams), since UML is the standard, comprehensive technique for visualizing and documenting a software system's structure and behavior for developers to implement.